



Chromatics AI

Technical Assignment

Date: August, 2026

Dear Candidate,

Thank you for taking the time to speak with us. As the next step, we'd like you to work through a short take-home assignment that reflects the kind of ambiguous, real-world problem our frontend team deals with regularly.

Background

Alo Relief Trust (fictional) is an NGO running health, education, and disaster-relief programs across several regions. Its field teams have accumulated roughly 100,000 documents — enrollment forms, medical intake sheets, ID scans, handwritten notes, and more — spread across drives, email, and scanned folders. Alo wants to digitize this archive: upload it, track its processing, and eventually browse the extracted information as structured records. They've asked us to prototype the frontend for this.

The Problem

Design and build a frontend prototype that lets someone:

- upload documents, one at a time or in bulk
- understand that a real batch could be very large — up to roughly 100,000 files
- see upload and processing progress
- tell which files are pending, processing, completed, or failed
- view the normalized information extracted from processed files
- inspect an individual result in more detail
- do something sensible when a file fails or produces incomplete or uncertain data

This is intentionally not fully specified. We haven't defined the exact data model, workflow, or layout — make reasonable assumptions, decide what matters most for someone using this day to day, and document the decisions you make. A documented assumption is a perfectly good answer to an unclear point.

What You Don't Need to Build

This is a frontend prototype, not a production platform. You do not need to:

- actually, upload or store 100,000 real files
- build real OCR or a real ML/AI pipeline
- build a production backend

You're free to simulate scale, latency, and processing however you like — mock data, generated fixtures, artificial delays, and randomized outcomes are all fair game.





Chromatics AI

Approach

Pick whichever combination best demonstrates your thinking:

- fully mocked / fixture-based API responses
- processing simulated entirely on the frontend
- a small mock backend, if you want one
- a real AI or OCR API/model, if you want to demonstrate real normalization
- any mix of the above

Using AI is entirely optional — it is not required, and it is not worth extra credit on its own.

A Normalized Record (Example Only)

A processed document might eventually produce something like the fields below. Feel free to define a different schema if it serves the product better — this is just to illustrate the idea:

```
documentId, personName, phone, location, programName,  
date, documentType, confidence, processingStatus
```

Expect these values to sometimes be missing, inconsistent, or uncertain — the interface should make that reality visible rather than hide it.

What We're Interested In

How you approach structuring a frontend for a problem like this — component architecture, state management, async workflows, large datasets, loading/empty/error states, retries and partial failures, performance, accessibility, and how clearly you communicate trade-offs. You don't need to demonstrate all of these — a well-scoped, thoughtfully engineered slice is worth more to us than a large, unfinished one.

Timeline

We'd like this back by end of day Friday. If that timing doesn't work for you, just let us know — we'd rather hear that than receive something rushed.

Deliverables

- your source code, as a Git repository (or a zip if that's easier)
- instructions to run it locally
- a short README covering the assumptions you made, the key technical decisions and why, and what you'd improve with more time
- optional: a few screenshots or a short screen recording

We're looking for good judgment and clear thinking more than feature count — a smaller, well-reasoned implementation beats a large, unfinished one. If anything above is unclear, feel free to reach out.

Timeline

This assignment is open from

